GUIA COMPLETO DE ESTUDO

Observatório SQL

Análise e otimização de consultas SQL com base em histórico de execução e planos do PostgreSQL

Este material explica o projeto do zero: problema, banco, carga experimental, métricas, EXPLAIN, regras heurísticas, interface e apresentação do TCC.

Versão de estudo baseada no ambiente validado em 13/10/2026
PostgreSQL 18.6 | Python 3.14.7 | Interface web local

Como estudar este guia

Leia primeiro os capítulos 1 a 4 para construir o modelo mental. Depois estude os casos Q05 e Q09, que formam a melhor história para a apresentação. Use o glossário e as perguntas finais para revisar.

> **A frase que resume o projeto**
> Meu projeto identifica consultas que consomem tempo no banco, examina o plano escolhido pelo PostgreSQL e produz hipóteses de melhoria que depois precisam ser medidas.

Sumário

Como estudar este guia 2

Sumário . . . 2

1. Visão geral do projeto 4

Problema de pesquisa . . . 4

Objetivo prático . . . 4

O que ele não faz . . . 4

2. Arquitetura e fluxo de dados 5

Caminho de uma medição . . . 5

Componentes do código . . . 5

3. Banco de dados experimental 6

Modelo relacional simplificado . . . 6

Por que as restrições importam . . . 6

Distribuição controlada . . . 6

4. Método experimental 7

A rodada atual . . . 7

Reprodutibilidade . . . 7

5. As 12 consultas do workload 8

Como interpretar . . . 8

6. Histórico real e pg_stat_statements 9

O filtro do coletor . . . 9

Por que o histórico pode zerar . . . 9

7. EXPLAIN e leitura de planos 10

Custo não é tempo . . . 10

Linhas estimadas . . . 10

8. Regras do analisador preventivo 11

Cálculo da atenção . . . 11

Por que chamar de heurística . . . 11

9. Estudo de caso Q05 - pedidos de 2026 12

Por que ela recebe 9 pontos . . . 12

Alternativa investigada . . . 12

10. Estudo de caso Q09 - função e índice 13

Validação medida . . . 13

Condição de equivalência . . . 13

11. Interface Observatório SQL 14

Ações importantes . . . 14

Proteções do servidor local . . . 14

12. Como adicionar uma nova consulta 15

1. Cadastrar no workload . . . 15

2. Cadastrar a descrição visual . . . 15

3. Gerar histórico . . . 15

4. Conferir a ciência . . . 15

13. Limitações e validade 16

Ameaças à validade . . . 16

Como responder à banca . . . 16

14. Roteiro de apresentação 17

Perguntas prováveis . . . 17

15. Operação e solução de problemas 18

Uso cotidiano . . . 18

Comandos de diagnóstico . . . 18

16. Glossário essencial 19

Três frases para memorizar . . . 19

17. Revisão e autoavaliação 20

Respostas de conferência . . . 20

Fim do guia . . . 20

1. Visão geral do projeto

O projeto é um protótipo experimental para analisar desempenho de consultas SQL. Ele não tenta adivinhar uma consulta perfeita. Ele reúne evidências para indicar onde vale investigar.

Problema de pesquisa

Uma consulta pode estar correta e ainda consumir muito tempo, processar dados demais ou induzir o planejador a estimar volumes errados. Em um sistema real, olhar apenas para o SQL não mostra toda a história: é preciso observar o que já ocorreu no servidor e como o banco planeja responder.

Objetivo prático

- Gerar uma carga controlada com consultas de perfis diferentes.
- Coletar chamadas, tempos e linhas acumuladas por consulta.
- Ordenar consultas por tempo total, tempo médio e frequência.
- Solicitar o plano estimado de um SELECT sem executar a consulta candidata.
- Aplicar regras explícitas e gerar alertas explicáveis.
- Comparar uma consulta com uma alternativa e, em um experimento separado, medir a hipótese.

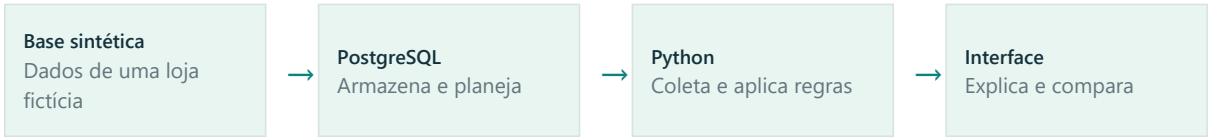
O que ele não faz

O protótipo não cria índices, não altera tabelas, não reescreve SQL automaticamente e não usa inteligência artificial. A pontuação de atenção não é uma probabilidade de erro. É apenas a soma limitada a 10 dos pontos atribuídos pelas regras.

> **Modelo mental**
> Histórico responde: o que aconteceu? Plano responde: como o PostgreSQL pretende trabalhar? Regras respondem: o que merece investigação? Experimento responde: a hipótese realmente melhorou o tempo?

2. Arquitetura e fluxo de dados

O sistema tem quatro partes principais. Cada uma tem uma responsabilidade simples e separada.



Caminho de uma medição

1. O gerador lê as 12 consultas de sql/queries.sql.
2. A carga executa cada consulta a quantidade configurada, totalizando 730 chamadas.
3. O PostgreSQL acumula métricas na extensão pg_stat_statements.
4. O coletor lê as métricas agregadas e monta os rankings.
5. O analisador pede EXPLAIN com ANALYZE FALSE, obtém o plano JSON e aplica regras.
6. A interface combina histórico, plano e explicações em telas didáticas.

Componentes do código

Arquivo ou pasta	Responsabilidade
sql/schema.sql	Cria as cinco tabelas e suas restrições.
sql/queries.sql	Catálogo das 12 consultas e número de chamadas.
scripts/populate_database.py	Gera e insere a base sintética.
scripts/generate_workload.py	Executa a carga controlada em modo somente leitura.
src/collector.py	Lê pg_stat_statements e filtra o escopo experimental.
src/analyzer.py	Valida um SELECT e solicita seu plano estimado.
src/plan_analyzer.py	Aplica as regras heurísticas e calcula a atenção.
src/webapp.py + web/	Servidor local e interface visual.
tests/test_webapp.py	Testa o contrato HTTP e as barreiras de segurança.

3. Banco de dados experimental

A base imita uma loja. Os dados são sintéticos, mas são registros reais armazenados no PostgreSQL. Isso permite repetir experimentos sem usar dados pessoais.

Modelo relacional simplificado

> **Relacionamentos**
> clientes 1:N pedidos; pedidos 1:N itens_pedido; produtos 1:N itens_pedido; pedidos 1:N pagamentos.

Tabela	Papel	Registros validados	Chaves e restrições principais
clientes	Quem compra	100.000	PK id; email único; estado com 2 letras.
produtos	Itens vendidos	50.000	PK id; preço e estoque não negativos.
pedidos	Compra do cliente	1.000.000	FK cliente_id; status controlado; valor não negativo.
itens_pedido	Produtos de cada pedido	2.498.714	FK pedido_id e produto_id; quantidade positiva.
pagamentos	Pagamento do pedido	1.000.000	FK pedido_id; forma e status controlados.

Por que as restrições importam

Chaves primárias permitem identificar registros. Chaves estrangeiras mantêm as relações válidas. CHECKs impedem valores fora das regras definidas. O índice único criado para clientes.email também é essencial no estudo de caso Q09.

Distribuição controlada

A população foi planejada para produzir volumes e distribuições úteis ao experimento. Na validação, os cinco indicadores de inconsistência retornaram zero e os clientes frequentes concentraram 80,02% dos pedidos. Essa assimetria ajuda a criar consultas com comportamentos diferentes.

> **Cuidado operacional**
> A base já está populada. Não execute novamente o gerador de dados no banco atual. Para atualizar somente o histórico, use Gerar historico.cmd.

4. Método experimental

O método separa claramente carga, coleta, análise preventiva e validação. Essa separação é importante para não confundir estimativas com medições.

Fase	Entrada	Ação	Saída
Preparação	Schema e parâmetros	Cria e popula a base	Dados sintéticos validados
Carga	12 SELECTs	Executa 730 chamadas	Métricas acumuladas no servidor
Coleta	pg_stat_statements	Filtra e ordena consultas	Rankings e JSON/CSV
Análise	Um SELECT	EXPLAIN sem ANALYZE + regras	Alertas e pontuação 0 a 10
Comparação	Original e alternativa	Compara planos estimados	Hipótese de mudança
Validação	Duas versões equivalentes	Execuções intercaladas	Tempos e resultados medidos

A rodada atual

Em 13/10/2026, a carga completou 730 execuções sem erro. O processo cliente levou 52,6 s e o PostgreSQL acumulou aproximadamente 51,65 s de tempo de execução. Esses números são diferentes porque o cliente também mede comunicação, transferência e consumo dos resultados.

Reprodutibilidade

- A ordem das chamadas usa uma semente pseudoaleatória conhecida.
- A sessão da carga é somente leitura e possui limite de tempo.
- As métricas são coletadas para o mesmo banco, usuário e consultas de primeiro nível.
- O relatório registra versões, comandos, consultas e resultados.
- Cache, concorrência, hardware e configuração podem mudar tempos entre rodadas.

> **Regra científica**
> Uma diferença de plano é evidência para formular uma hipótese. A conclusão sobre desempenho exige medir versões semanticamente equivalentes sob condições controladas.

5. As 12 consultas do workload

As frequências foram escolhidas para criar uma mistura: consultas rápidas e frequentes convivem com consultas raras e caras. Por isso, olhar apenas para a quantidade de chamadas não basta.

ID	Chamadas	Tema	O que exercita
Q01	500	Cliente por chave	Busca seletiva pela chave primária.
Q02	100	Produtos e preço	Filtros, ordenação e LIMIT.
Q03	60	Pedidos do cliente	Filtro por FK e ordenação por data.
Q04	20	Cancelados	SELECT * e retorno amplo.
Q05	15	Pedidos de 2026	JOIN, função na data e grande retorno.
Q06	10	Vendas por região	Cinco tabelas, agregação e ordenação.
Q07	8	Resumo mensal	Intervalo de datas, agrupamento e DATE_TRUNC.
Q08	6	Maiores pedidos	Ordenação ampla com LIMIT 1000.
Q09	5	Busca por e-mail	LOWER aplicado sobre coluna indexada.
Q10	3	Acima da média	Subconsulta e resultado volumoso.
Q11	2	Contagem por cliente	Subconsulta correlacionada.
Q12	1	Todos os itens	Leitura completa de 2.498.714 registros.

Como interpretar

Q01 domina a frequência, mas cada chamada é barata. Q05, Q06 e Q12 têm poucas chamadas, porém processam grandes volumes. O ranking por tempo total ajuda a priorizar onde o banco gastou mais no conjunto da rodada.

> **Impacto acumulado**
>
> No coletor, impact_score é igual a tempo médio vezes número de chamadas. Como pg_stat_statements já fornece o tempo total, os dois representam o mesmo princípio: custo por chamada multiplicado pela frequência.

6. Histórico real e pg_stat_statements

pg_stat_statements é uma extensão do PostgreSQL que agrupa consultas de forma normalizada e acumula métricas. Valores literais podem aparecer como parâmetros, permitindo que várias execuções da mesma forma de SQL compartilhem uma entrada.

Métrica	Significado	Pergunta que ajuda a responder
calls	Quantidade acumulada de chamadas	Esta consulta acontece com frequência?
total_exec_time	Soma do tempo no servidor	Onde o banco gastou mais tempo no total?
mean_exec_time	Tempo médio por chamada	Cada execução costuma ser cara?
rows	Linhas devolvidas no acumulado	Quanto resultado foi entregue?
queryid	Identificador da forma normalizada	Qual histórico corresponde ao plano analisado?

O filtro do coletor

O coletor seleciona SELECTs do banco tcc_sql_optimizer, do usuário atual, em primeiro nível, que referenciam as cinco tabelas públicas do experimento. Consultas administrativas e verificações internas são excluídas. Ele não reseta as estatísticas.

Por que o histórico pode zerar

As estatísticas podem ser reiniciadas por comando, reinstalação, alteração de arquivos ou comportamento do serviço. Em 13/10, o histórico estava vazio após a inicialização do PostgreSQL e precisou de uma nova rodada. O arquivo Gerar historico.cmd executa novamente somente o workload, sem repopular as tabelas.

> **Histórico não é log individual**
>
> Você vê totais agregados por forma de consulta. Não vê cada chamada separadamente, seus parâmetros individuais ou uma linha temporal detalhada.

7. EXPLAIN e leitura de planos

EXPLAIN pergunta ao PostgreSQL qual estratégia ele pretende usar. Neste projeto, ANALYZE fica FALSE; portanto, o SELECT candidato não é executado. O resultado é solicitado em JSON para que o Python percorra cada nó.

```
EXPLAIN (ANALYZE FALSE, VERBOSE TRUE, COSTS TRUE,
         SETTINGS TRUE, SUMMARY TRUE, FORMAT JSON)
SELECT ...;
```

Termo	Tradução prática	Leitura cuidadosa
Seq Scan	Percorrer registros da tabela	Pode ser ruim com filtro seletivo em tabela grande; pode ser adequado se grande parte será lida.
Index Scan	Localizar por um índice	Útil em buscas seletivas; também tem custo de acessar tabela e índice.
Hash Join	Combinar tabelas com tabela de busca	Comum ao juntar conjuntos grandes por igualdade.
Nested Loop	Repetir busca do lado interno	Ótimo para poucos registros; pode crescer muito com volumes altos.
Sort	Ordenar registros	Pode consumir memória e disco quando o volume é alto.
Aggregate	Agrupar ou resumir	Produz COUNT, SUM, AVG e grupos.
Gather	Reunir trabalho paralelo	Combina os resultados produzidos por processos paralelos.

Custo não é tempo

Total Cost é uma unidade interna para o planejador comparar alternativas. Ele mistura estimativas de CPU e entrada/saída de acordo com parâmetros do PostgreSQL. Não deve ser apresentado como milissegundos.

Linhas estimadas

Plan Rows indica quantas linhas o planejador espera em cada etapa. Estimativas muito distantes da realidade podem levar a escolhas inadequadas de junção, acesso e ordenação. O projeto compara a saída estimada com a média histórica de linhas por chamada.

8. Regras do analisador preventivo

As regras são determinísticas e explicáveis. Cada alerta inclui severidade, evidência encontrada e uma recomendação de investigação.

Regra	Condição resumida	Pontos
Seleção ampla	SQL contém SELECT *.	1
Varredura sequencial	Seq Scan com filtro em relação de pelo menos 50.000 linhas.	3
Função no filtro	Filtro contém extract, lower, upper, date_trunc, substring ou trim.	2
Ordenação volumosa	Sort estima pelo menos 10.000 linhas.	2
Nested Loop amplo	Produto estimado dos dois primeiros ramos chega a 1.000.000.	2
Resultado volumoso	Nó principal estima pelo menos 100.000 linhas.	3
Divergência	Estimativa e média histórica diferem por fator de pelo menos 10.	3
Subplano	O plano contém nó associado a SubPlan.	2

Cálculo da atenção

Os pontos dos achados únicos são somados e limitados a 10. De 0 a 2, a atenção é baixa; de 3 a 6, média; de 7 a 10, alta. Um resultado 0 não prova que a consulta é rápida. Um resultado 10 não prova que está errada.

Por que chamar de heurística

Uma heurística é uma regra prática. Ela captura padrões que frequentemente merecem investigação, mas depende do contexto. Por exemplo, Seq Scan não é um defeito automático: ler a tabela inteira pode ser a melhor escolha para retornar metade dos registros.

> **Valor acadêmico**
> Como as regras estão explícitas no código, é possível explicar, reproduzir, criticar e evoluir cada decisão. Isso é mais importante para o protótipo do que fingir certeza.

9. Estudo de caso Q05 - pedidos de 2026

A Q05 junta pedidos e clientes e aplica EXTRACT sobre a data. Ela é o melhor exemplo para explicar divergência de cardinalidade.

```
SELECT *
FROM public.pedidos p
JOIN public.clientes c ON c.id = p.cliente_id
WHERE EXTRACT(YEAR FROM p.data_pedido) = 2026;
```

Evidência	Valor observado
Chamadas históricas	15
Linhas médias por chamada	499.676
Linhas estimadas pelo plano	5.000
Subestimação aproximada	99,9 vezes
Nó principal	Gather
Custo estimado	19.003,00 unidades
Atenção	9/10 - alta

Por que ela recebe 9 pontos

- SELECT *: 1 ponto.
- Seq Scan com filtro em pedidos, que possui cerca de 1 milhão de linhas: 3 pontos.
- EXTRACT aplicado à coluna data_pedido: 2 pontos.
- Diferença superior a 10 vezes entre estimativa e histórico: 3 pontos.

Alternativa investigada

```
WHERE p.data_pedido >= TIMESTAMP '2026-01-01 00:00:00'
  AND p.data_pedido <  TIMESTAMP '2027-01-01 00:00:00'
```

O intervalo compara a coluna diretamente e melhora a estimativa para cerca de 498.587 linhas. Mesmo assim, como quase metade dos pedidos pertence a 2026, uma varredura pode continuar apropriada. O custo estimado maior da alternativa não significa automaticamente pior desempenho: ele pode refletir uma estimativa mais honesta do volume.

> **O que defender**
>
> O resultado mais forte da Q05 não é prometer um índice. É mostrar que a expressão original dificultou a estimativa de cardinalidade e que a forma por intervalo permite ao planejador enxergar melhor a quantidade de linhas.

10. Estudo de caso Q09 - função e índice

A coluna clientes.email possui restrição UNIQUE, que cria um índice. A consulta original aplica LOWER à coluna, mudando a expressão que o índice representa.

Original	Alternativa
`WHERE LOWER(email) = 'cliente123@example.com'`	`WHERE email = 'cliente123@example.com'`
Seq Scan	Index Scan
500 linhas estimadas	1 linha estimada
Custo 2.868,00	Custo 8,44
Atenção 8/10	Atenção 0/10

Validação medida

Depois da análise preventiva, 100 chamadas de cada versão foram executadas de forma intercalada. Os primeiros resultados foram iguais e cada chamada devolveu uma linha.

Versão	Chamadas	Tempo médio no servidor	Redução
Original com LOWER	100	26,3300 ms	-
Comparação direta	100	0,0303 ms	99,88%

A alternativa foi aproximadamente 869 vezes mais rápida no tempo médio do servidor nesta rodada. O cliente Python mediu redução de 98,52%, pois inclui comunicação, driver e controle do laço.

Condição de equivalência

Remover LOWER só é equivalente porque o gerador salva todos os e-mails em minúsculas. Se o sistema aceitasse capitalização variável, a alternativa poderia devolver outro resultado. Outras soluções seriam normalização na escrita, tipo citext ou índice de expressão, cada qual com custos e requisitos próprios.

> **Cadeia de evidências**
>
> Regra detectou função e Seq Scan -> comparação previu Index Scan -> experimento confirmou equivalência neste conjunto -> medição mostrou redução de 99,88%.

11. Interface Observatório SQL

A interface traduz os resultados técnicos sem esconder as limitações. Ela roda apenas em 127.0.0.1:8765 e usa o mesmo banco e os mesmos módulos Python do terminal.

Tela	O que mostra	Como usar na apresentação
Visão geral	12 consultas, 730 chamadas, tempo acumulado, tabelas e ranking.	Apresente a base e mostre onde o banco gastou mais.
Analisar consulta	SQL, plano, histórico, pontuação e achados.	Use Q05 para explicar estimativa versus medição.
Comparar versões	Dois planos lado a lado.	Use o caso de e-mail para mostrar Seq Scan e Index Scan.
Entenda o projeto	Conceitos, fluxo e roteiro.	Use como apoio se esquecer algum termo.

Ações importantes

- Abrir interface.cmd inicia o servidor e abre o navegador. A janela deve permanecer aberta.
- Atualizar leitura consulta novamente as métricas; não executa o workload.
- Analisar consulta aceita um SELECT e não executa o resultado.
- Comparar planos também usa somente estimativas.
- Os botões de download salvam JSON com o que está sendo exibido.

Proteções do servidor local

O backend limita o host ao endereço local, exige um cabeçalho próprio nas operações POST, verifica origem, restringe arquivos estáticos e aceita no máximo 64 KB de JSON. A validação rejeita comandos que não começam por SELECT, múltiplas instruções e SELECT INTO. A conexão de análise fica em modo somente leitura.

> **Teste confirmado**
> Sete testes automatizados foram aprovados. Também foram verificados SQL inválido, recuperação após erro, consulta sem histórico, download JSON e layout móvel. Analisar e comparar mantiveram as 730 chamadas inalteradas.

12. Como adicionar uma nova consulta

Colar um SELECT na tela permite analisar seu plano, mas não cria histórico porque a consulta candidata não é executada. Para transformá-la em exemplo permanente, há duas etapas distintas: cadastrar e medir.

1. Cadastrar no workload

```
-- name: Q13 | calls: 10 | title: Pedidos entregues de alto valor
SELECT id, cliente_id, valor_total
FROM public.pedidos
WHERE status = 'entregue'
  AND valor_total > 1000;
```

O identificador deve ser único. calls define quantas vezes a carga executará o SELECT. O bloco deve conter uma única instrução terminada por ponto e vírgula.

2. Cadastrar a descrição visual

No dicionário DESCRIPTIONS de src/webapp.py, associe Q13 a uma frase curta que explique o propósito. Sem isso, a lista de exemplos não terá uma explicação apropriada. A interface atual não possui um formulário para persistir esse cadastro.

3. Gerar histórico

Execute Gerar historico.cmd e depois clique em Atualizar leitura. O script soma as novas medições ao histórico existente; não apaga medições anteriores. Uma consulta executada por DBeaver também pode aparecer se estiver no escopo do coletor, mas será chamada de Consulta fora dos exemplos quando o Query ID não corresponder ao catálogo.

4. Conferir a ciência

- Defina por que a nova consulta representa um cenário relevante.
- Registre os dados e parâmetros usados.
- Não compare tempos de rodadas com condições diferentes sem declarar a limitação.
- Se criar uma alternativa, comprove primeiro a equivalência dos resultados.

> **Diferença essencial**
> Cadastrar dá nome e descrição. Executar produz histórico. Analisar produz plano. São três operações separadas.

13. Limitações e validade

Um bom TCC não esconde limitações; ele delimita o que suas evidências permitem concluir.

Limitação	Implicação
Base sintética	Os resultados não representam automaticamente um sistema de produção.
Uma máquina e uma configuração	Tempos variam com hardware, cache, memória e parâmetros.
Métricas agregadas	Não há parâmetros ou duração de cada chamada individual.
Plano sem ANALYZE	Não contém tempo real nem contagens reais daquela análise.
Regras heurísticas	Alertas indicam investigação; não provam causa nem solução.
Poucas regras	Padrões importantes podem não ser detectados.
Equivalência contextual	Uma reescrita válida na base sintética pode mudar resultados em outro domínio.
Histórico acumulado	Nova rodada é somada se as estatísticas não forem resetadas.

Ameaças à validade

- Validade interna: ordem de execução, aquecimento de cache e processos concorrentes podem influenciar a medição.
- Validade externa: o conjunto de consultas e dados representa um domínio específico.
- Validade de construção: custo estimado, tempo medido e atenção heurística são conceitos diferentes.
- Validade de conclusão: uma rodada muito pequena pode produzir resultado instável.

Como responder à banca

Se perguntarem se o sistema garante otimização, responda: não. Ele prioriza consultas e gera hipóteses explicáveis. A validação controlada continua necessária. O caso Q09 mostra uma hipótese que foi posteriormente confirmada, mas dentro das condições documentadas.

14. Roteiro de apresentação

Roteiro sugerido para uma demonstração de cinco a sete minutos.

1. Problema: uma consulta pode funcionar e ainda desperdiçar tempo. Quero localizar evidências de consultas que merecem atenção.
2. Ambiente: mostre as cinco tabelas da loja sintética e diga que a rodada possui 730 execuções de 12 formas de consulta.
3. Histórico: abra Visão geral e explique frequência, tempo total e tempo médio. Mostre que consulta frequente não é necessariamente a mais cara por chamada.
4. Análise preventiva: abra Q05. Mostre 5.000 linhas estimadas contra 499.676 históricas e os quatro achados que somam 9/10.
5. Comparação: abra a busca por e-mail. Mostre Seq Scan contra Index Scan e custo 2.868 contra 8,44.
6. Validação: conte que 100 chamadas de cada versão foram medidas e a média do servidor caiu 99,88%, com resultados equivalentes na base.
7. Limite: ressalte que o protótipo sugere hipóteses e não aplica mudanças automaticamente.

Perguntas prováveis

Pergunta	Resposta curta
Por que não usar EXPLAIN ANALYZE na tela?	Para não executar uma consulta desconhecida durante a análise preventiva. A medição fica em experimento controlado.
Seq Scan é sempre ruim?	Não. Pode ser correto quando grande parte da tabela será lida.
Custo 19.003 significa 19 segundos?	Não. Custo é uma unidade interna, não tempo.
A pontuação é inteligência artificial?	Não. É a soma de regras explícitas em Python.
Por que a Q09 melhorou?	A comparação direta permitiu usar o índice único de email neste conjunto normalizado.
O sistema otimiza sozinho?	Não. Ele explica sinais e apoia uma decisão experimental.

15. Operação e solução de problemas

Uso cotidiano

- Dê dois cliques em Abrir interface.cmd.
- Mantenha a janela do servidor aberta durante o uso.
- Comece por Entenda o projeto e siga para Q05 e Q09.
- Se o histórico estiver vazio, use Gerar historico.cmd uma vez e atualize a leitura.

Comandos de diagnóstico

```
# Testar conexão
.\.venv\Scripts\python.exe -m src.database

# Abrir a interface
.\.venv\Scripts\python.exe -m src.webapp --open

# Analisar o arquivo Q05 sem executar o SELECT
.\.venv\Scripts\python.exe -m src.analyzer --sql-file sql/candidate_query.sql

# Ler o histórico no terminal
.\.venv\Scripts\python.exe -m src.collector --top 5 --no-export

# Rodar testes
.\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Sintoma	Causa provável	Ação
Banco indisponível	Serviço parado ou .env incorreto	Confira postgresql-x64-18 e as variáveis locais.
Página não abre	Servidor da interface fechado	Reabra Abrir interface.cmd.
Histórico zerado	Estatísticas reiniciadas	Execute Gerar historico.cmd; não repopule o banco.
SQL rejeitado	Comando não é um SELECT único	Use uma instrução SELECT, sem WITH ou SELECT INTO.
Sem histórico correspondente	Consulta nunca foi executada ou Query ID mudou	O plano ainda é válido; execute carga controlada se precisar de medição.
Porta 8765 ocupada	Outra aplicação está usando a porta	Feche o processo responsável ou inicie src.webapp com outra porta.

16. Glossário essencial

Termo	Definição
Cardinalidade	Quantidade de linhas esperada ou observada em uma etapa.
Consulta SQL	Instrução enviada ao banco para ler ou modificar dados; a interface aceita somente SELECT.
Custo estimado	Unidade interna usada pelo planejador para comparar estratégias.
EXPLAIN	Comando que mostra o plano escolhido pelo PostgreSQL.
EXPLAIN ANALYZE	Executa a consulta e anexa medições reais; não é usado na análise preventiva da interface.
Heurística	Regra prática que aponta um padrão, sem garantia universal.
Índice	Estrutura auxiliar que acelera certos acessos em troca de espaço e custo de manutenção.
Query ID	Identificador calculado pelo PostgreSQL para uma forma normalizada de consulta.
Seletividade	Proporção de linhas que satisfaz um filtro; filtro seletivo retorna pequena parte da tabela.
Tempo do cliente	Inclui servidor, comunicação, driver e consumo do resultado.
Tempo do servidor	Tempo de execução acumulado pelo PostgreSQL, sem toda a sobrecarga do cliente.
Workload	Conjunto planejado de consultas e frequências usado no experimento.

Três frases para memorizar

1

Custo estimado não é tempo medido.

2

Um plano diferente cria uma hipótese; a medição controlada sustenta a conclusão.

3

Seq Scan, índice e pontuação só fazem sentido junto do volume, da seletividade e da regra de negócio.

17. Revisão e autoavaliação

1. Qual problema o projeto tenta resolver?
2. Qual a diferença entre histórico, plano e validação?
3. Por que pg_stat_statements não é um log individual?
4. Por que custo estimado não pode ser apresentado como milissegundos?
5. Quais quatro regras aparecem na Q05 e como formam 9 pontos?
6. Por que a forma por intervalo melhora a estimativa da Q05?
7. Por que remover LOWER permite Index Scan na Q09?
8. Qual condição torna as duas formas da Q09 equivalentes?
9. Por que analisar uma consulta na interface não a coloca no histórico?
10. Como cadastrar e medir uma Q13?
11. Que fatores limitam a generalização dos resultados?
12. O que você responderia se a banca perguntasse se o sistema otimiza automaticamente?

Respostas de conferência

1) Priorizar e investigar consultas potencialmente custosas. 2) Histórico mede o passado agregado; plano estima a estratégia; validação executa versões controladas. 3) Porque agrupa chamadas normalizadas. 4) Porque custo é unidade interna. 5) SELECT *, Seq Scan, função no filtro e divergência: 1+3+2+3. 6) Compara a coluna diretamente e melhora a estimativa de cardinalidade. 7) A expressão passa a corresponder à coluna indexada. 8) E-mails da base estão em minúsculas. 9) Porque usa EXPLAIN com ANALYZE FALSE. 10) Adicionar bloco em queries.sql, descrição no backend, executar carga e atualizar. 11) Dados sintéticos, uma máquina, cache, configuração, agregação e heurísticas. 12) Não; o protótipo formula hipóteses que precisam ser validadas.

> **Você está pronto para apresentar quando**
>
> Consegue explicar Q05 e Q09 sem ler, distingue custo de tempo e sabe defender por que as recomendações são hipóteses, não decisões automáticas.

Fim do guia

Documentação consolidada a partir do código, dos registros experimentais e da interface do projeto. Valores de tempo pertencem às rodadas documentadas e podem variar em novas execuções.